

Лабораторная работа №26. Современный JavaScript

Цель работы:

- Понять основы работы с REST API;
- Подготовиться к изучению React и TypeScript.

Подготовка проекта

1. Откройте **Visual Studio Code**
2. Откройте **предыдущий** проект.
3. Создайте его копию, с новым именем:

Lab26_ModernJS

4. Откройте созданную папку в **Visual Studio Code**

Инициализация Git-репозитория

1. Удалите скрытую папку **.git** из репозитория.
2. Инициализируйте Git-репозиторий: **git init**
3. Добавьте файлы в индекс: **git add .**
4. Создайте первый коммит: **git commit -m "Initial commit"**

Шаг 13. Итоговое задание - Простой Менеджер задач

В этом шаге мы создадим полноценное приложение, используя все изученные концепции.

13.1. Упрощенная HTML-структура

Откройте **index.html**. Добавим внутрь тега **<body>**.

1. Изменим **заголовок первого и второго уровня**:

```
10 <div class="container">
11 <h1>Lab 23-25 - Modern JavaScript</h1>
12 <h2>Менеджер задач</h2>
```

2. Добавим блок формы для ввода новой задачи (класс **task-form**):

```
12 <h2>Менеджер задач</h2>
13
14 <div class="task-form">
15 <h3>Добавить новую задачу</h3>
16 <input type="text" id="taskInput" placeholder="Введите
задачу ..." />
17 <button id="addTask">Добавить</button>
18 </div>
```

3. Добавим блок фильтров задач (класс **filters**):

```

20     <div class="filters">
21         <button class="filter-btn active" data-filter="all">Все</button>
22         <button class="filter-btn" data-filter="active">Активные</button>
23         <button class="filter-btn" data-filter="completed">Завершённые</
button>
24     </div>

```

4. Добавим **контейнер** для отображения списка задач:

```

26     <div id="taskList" class="task-list"></div>

```

5. Добавим блок статистики (класс **stats**):

```

28     <div class="stats">
29         <p>Всего задач: <span id="totalTasks">0</span></p>
30         <p>Выполнено: <span id="completedTasks">0</span></p>
31     </div>

```

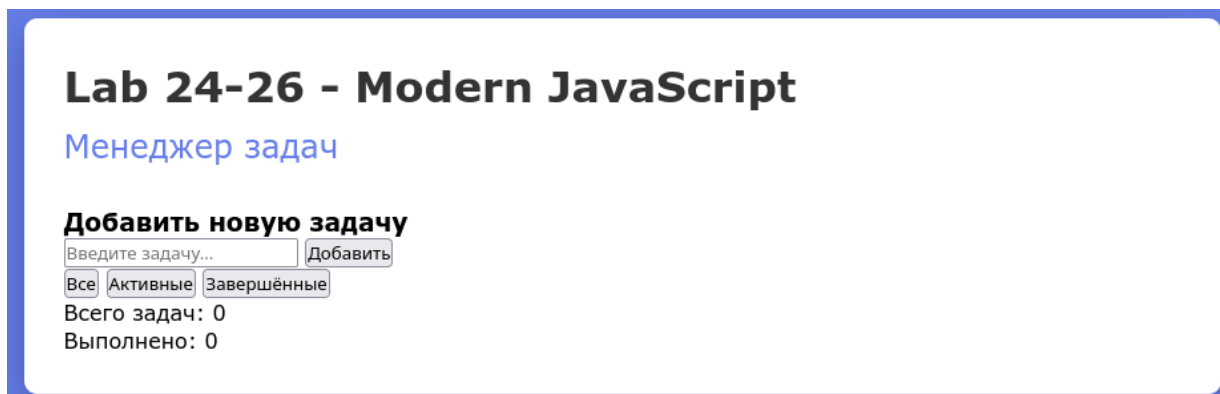
6. Изменим путь к скрипту:

```

23     <script type="module" src="js/api.js"></script>

```

Смотрим результат: Откройте страницу в браузере через **Live Server**:



Шаг 1: Создаём класс TaskManager

Откройте файл **js/app.js** Создадим основу нашего приложения - класс для управления задачами:

```

1  // Модуль управления задачами
2  class TaskManager {
3      constructor() {
4          this.tasks = this.loadTasks(); // Загружаем задачи из памяти браузера
5          this.currentFilter = "all"; // По умолчанию показываем все задачи
6          this.init(); // Запускаем инициализацию
7      }
8
9      init() {
10         this.renderTasks(); // Отображаем задачи на странице
11         this.attachEventListeners(); // Подключаем обработчики событий
12     }
13 }

```

Что делает этот код:

- Создаём класс `TaskManager` для управления всеми задачами
- В конструкторе загружаем сохранённые задачи
- Устанавливаем фильтр по умолчанию ("all" - показывать все)
- Метод `init()` запускает отрисовку и подключает события

Шаг 2: Загрузка и сохранение задач

Добавим методы для работы с `localStorage` (хранилище браузера) **внутри класса**:

```

14  // Загрузка задач из localStorage
15  loadTasks() {
16    const savedTasks = localStorage.getItem("tasks");
17    return savedTasks ? JSON.parse(savedTasks) : [];
18  }
19
20  // Сохранение задач в localStorage
21  saveTasks() {
22    localStorage.setItem("tasks", JSON.stringify(this.tasks));
23  }

```

Что делает этот код:

- `loadTasks()` - считывает задачи из памяти браузера. Если их нет, возвращает пустой массив
- `saveTasks()` - сохраняет текущий массив задач в память браузера в формате JSON

Шаг 3: Добавление новой задачи

Создадим метод для добавления задачи в список.

```

25  // Добавление новой задачи
26  addTask(text) {
27    if (!text?.trim()) return; // Проверяем, что текст не пустой
28    const task = {
29      id: Date.now(), // Уникальный ID на основе времени
30      text: text.trim(), // Убираем лишние пробелы
31      completed: false, // По умолчанию задача не выполнена
32      createdAt: new Date().toISOString(), // Дата создания
33    };
34    this.tasks.push(task); // Добавляем в массив
35    this.saveTasks(); // Сохраняем в localStorage
36    this.renderTasks(); // Обновляем отображение
37  }

```

Что делает этот код:

- Проверяем, что пользователь ввёл текст
- Создаём объект задачи с уникальным ID, текстом, статусом и датой

- Добавляем задачу в массив `this.tasks`
- Сохраняем изменения и обновляем интерфейс

Шаг 4: Переключение и удаление задач

Добавим методы для изменения статуса и удаления задач:

```
39  // Переключение статуса задачи (выполнена/не выполнена)
40  toggleTask(id) {
41    const task = this.tasks.find((t) => t.id === id);
42    if (task) {
43      // Меняем статус на противоположный
44      task.completed = !task.completed;
45      this.saveTasks();
46      this.renderTasks();
47    }
48  }

50  // Удаление задачи
51  deleteTask(id) {
52    // Удаляем задачу с указанным ID
53    this.tasks = this.tasks.filter((t) => t.id !== id);
54    this.saveTasks();
55    this.renderTasks();
56  }
```

Что делает этот код:

- **toggleTask()** - находит задачу по ID и меняет её статус на противоположный
 - **deleteTask()** - создаёт новый массив без удаляемой задачи
 - Оба метода сохраняют изменения и обновляют отображение

Шаг 5: Фильтрация задач

Создадим метод для фильтрации задач по статусу:

```
58  // Фильтрация задач
59  getFilteredTasks() {
60    switch (this.currentFilter) {
61      case "active":
62        // Только невыполненные
63        return this.tasks.filter((t) => !t.completed);
64      case "completed":
65        // Только выполненные
66        return this.tasks.filter((t) => t.completed);
67      default:
68        return this.tasks; // Все задачи
69    }
70  }
```

Что делает этот код:

- В зависимости от текущего фильтра возвращает нужные задачи
- "active" - только активные (невыполненные)
- "completed" - только завершённые
- "all" - все задачи

Шаг 6: Отрисовка задач на странице

Добавим метод для отображения задач в HTML:

```
// Отрисовка задач
renderTasks() {
  const taskList = document.getElementById("taskList");
  const filteredTasks = this.getFilteredTasks();
  if (filteredTasks.length === 0) {
    taskList.innerHTML = '<p style="text-align: center; color: #999; padding: 2rem;">Задач нет</p>';
  } else {
    taskList.innerHTML = filteredTasks
      .map(
        (task) => `
        <div class="task-item ${task.completed ? "completed" : ""}">
        <div class="task-content">
        <input
        type="checkbox"
        class="task-checkbox"
        ${task.completed ? "checked" : ""}
        data-id="${task.id}"
        >
        <span class="task-text">${task.text}</span>
        </div>
        <button class="task-delete" data-id="${task.id}">Удалить</button>
        </div>
        `
      )
      .join("");
  }
  this.updateStats(); // Обновляем статистику
}
```

Что делает этот код:

- Получаем отфильтрованные задачи
- Если задач нет - показываем сообщение
- Если есть - создаём HTML для каждой задачи с помощью map()
- Каждая задача включает: чекбокс, текст и кнопку удаления
- Вызываем обновление статистики

Шаг 7: Обновление статистики

Добавим метод для отображения количества задач:

```

102 | // Обновление статистики
103 | updateStats() {
104 |   document.getElementById("totalTasks").textContent =
      this.tasks.length;
105 |   document.getElementById("completedTasks").textContent =
      this.tasks.filter((t) => t.completed).length;
106 | }

```

Что делает этот код:

- Обновляет общее количество задач
- Считает и отображает количество выполненных задач

Шаг 8: Подключение обработчиков событий

Добавим метод для обработки действий пользователя:

```

// Подключение обработчиков событий
attachEventListeners() {
  // Добавление задачи
  document.getElementById("addTask").addEventListener("click", () => {
    const input = document.getElementById("taskInput");
    this.addTask(input.value);
    input.value = "";
  });

  // Добавление по Enter
  document.getElementById("taskInput").addEventListener("keypress", (e) => {
    if (e.key === "Enter") {
      this.addTask(e.target.value);
      e.target.value = "";
    }
  });

  // Делегирование событий для задач
  document.getElementById("taskList").addEventListener("click", (e) => {
    const id = parseInt(e.target.dataset.id);
    if (e.target.classList.contains("task-checkbox")) {
      this.toggleTask(id);
    } else if (e.target.classList.contains("task-delete")) {
      this.deleteTask(id);
    }
  });

  // Фильтры
  document.querySelectorAll(".filter-btn").forEach((btn) => {
    btn.addEventListener("click", (e) => {
      document.querySelectorAll(".filter-btn").forEach((b) => b.classList.remove("active"));
      e.target.classList.add("active");
      this.currentFilter = e.target.dataset.filter;
      this.renderTasks();
    });
  });
}

```

Что делает этот код:

- Добавляет обработчик на кнопку "Добавить"
- Позволяет добавлять задачи нажатием Enter
- Использует делегирование событий для чекбоксов и кнопок удаления
- Обрабатывает переключение фильтров

Шаг 9: Запуск приложения

В конце файла (**ЗА ПРЕДЕЛАМИ КЛАССА**) добавляем инициализацию:

```

143
144 // Инициализация приложения
145 const app = new TaskManager();
146

```

Что делает этот код:

- Создаёт экземпляр класса TaskManager
- Автоматически запускает конструктор и метод init()
- Приложение готово к работе!

Итоговый код скрипта:

```

1  // Модуль управления задачами
2  class TaskManager {
3  >  constructor() { ...
7  }
8
9  >  init() { ...
12 }
13
14 // Загрузка задач из localStorage
15 >  loadTasks() { ...
18 }
19
20 // Сохранение задач в localStorage
21 >  saveTasks() { ...
23 }
24
25 // Добавление новой задачи
26 >  addTask(text) { ...
37 }
38
39 // Переключение статуса задачи (выполнена/не выполнена)
40 >  toggleTask(id) { ...
47 }
48
49 // Удаление задачи
50 >  deleteTask(id) { ...
54 }
55
56 // Фильтрация задач
57 >  getFilteredTasks() { ...
66 }
67
68 // Отрисовка задач
69 >  renderTasks() { ...
96 }
97
98 // Обновление статистики
99 >  updateStats() { ...
102 }
103
104 // Подключение обработчиков событий
105 >  attachEventListeners() { ...
141 }
142 }
143
144 // Инициализация приложения
145 const app = new TaskManager();

```

Смотрим результат: Обновите страницу, добавьте несколько задач, затем обновите страницу снова.

Задачи должны остаться на месте! Они сохраняются в браузере!

13.3. Простые CSS-стили

Осталось только сделать красивее наше приложение. Откройте **styles.css** и обновите стили:

Сброс стандартных отступов и настройка box-sizing для всех элементов

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

Основные стили для страницы: шрифт, фон, минимальная высота и отступы

```
body {  
  font-family: Verdana, sans-serif;  
  background: #667eea;  
  min-height: 100vh;  
  padding: 20px;  
}
```

Контейнер для центрирования контента и создания карточки с тенью

```
.container {  
  max-width: 900px;  
  margin: 0 auto;  
  background: white;  
  padding: 30px;  
  border-radius: 10px;  
  box-shadow: 0 10px 30px rgba(0, 0, 0, 0.3);  
}
```

Стили для заголовка первого уровня

```
h1 {  
  color: #333;  
  margin-bottom: 10px;  
}
```

Стили для заголовка второго уровня (подзаголовка)

```
h2 {  
  color: #667eea;  
  margin-bottom: 30px;  
  font-weight: 400;  
}
```

Стили для заголовка третьего уровня

```
h3 {  
  color: #555;  
  margin-bottom: 15px;  
  font-size: 1.1rem;  
}
```

Форма добавления задачи. Стили для формы добавления новой задачи

```
.task-form {  
  margin-bottom: 25px;  
  padding: 20px;  
  background: #f8f9fa;  
  border-radius: 8px;  
}
```

Стили для поля ввода задачи

```
#taskInput {  
  width: 70%;  
  padding: 12px;  
  border: 2px solid #ddd;  
  border-radius: 5px;  
  font-size: 16px;  
  margin-right: 10px;  
}
```

Стили для поля ввода при фокусе

```
#taskInput:focus {  
  outline: none;  
  border-color: #667eea;  
}
```

Стили для кнопки добавления задачи

```
#addTask {  
  padding: 12px 25px;  
  background: #667eea;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  font-size: 16px;  
  cursor: pointer;  
  transition: background 0.3s;  
}
```

Стили для кнопки добавления задачи при наведении

```
#addTask:hover {  
  background: #5568d3;  
}
```

Фильтры. Стили для блока фильтров задач

```
.filters {  
  margin-bottom: 20px;  
  display: flex;  
  gap: 10px;  
}
```

Стили для кнопок фильтров

```
.filter-btn {  
  padding: 10px 20px;  
  border: 2px solid #667eea;  
  background: white;  
  color: #667eea;  
  border-radius: 5px;  
  cursor: pointer;  
  transition: all 0.3s;  
  font-size: 14px;  
}
```

Стили для кнопок фильтров при наведении

```
.filter-btn:hover {  
  background: #f0f0f0;  
}
```

Стили для активной кнопки фильтра

```
.filter-btn.active {  
  background: #667eea;  
  color: white;  
}
```

Список задач. Стили для контейнера списка задач

```
.task-list {  
  margin-bottom: 25px;  
  min-height: 200px;  
}
```

Стили для отдельного элемента задачи

```
.task-item {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 15px;
  margin-bottom: 10px;
  background: #f8f9fa;
  border-radius: 5px;
  border-left: 4px solid #667eea;
  transition: all 0.3s;
}
```

Стили для элемента задачи при наведении

```
.task-item:hover {
  background: #e9ecef;
  transform: translateX(5px);
}
```

Стили для выполненной задачи

```
.task-item.completed {
  opacity: 0.6;
  border-left-color: #28a745;
}
```

Стили для контента задачи (чекбокс + текст)

```
.task-content {
  display: flex;
  align-items: center;
  gap: 12px;
  flex: 1;
}
```

Стили для чекбокса задачи

```
.task-checkbox {
  width: 20px;
  height: 20px;
  cursor: pointer;
}
```

Стили для текста задачи

```
.task-text {  
  font-size: 16px;  
  color: #333;  
}
```

Стили для текста выполненной задачи

```
.task-item.completed .task-text {  
  text-decoration: line-through;  
  color: #999;  
}
```

Стили для кнопки удаления задачи

```
.task-delete {  
  padding: 8px 15px;  
  background: #dc3545;  
  color: white;  
  border: none;  
  border-radius: 5px;  
  cursor: pointer;  
  transition: background 0.3s;  
}
```

Стили для кнопки удаления задачи при наведении

```
.task-delete:hover {  
  background: #c82333;  
}
```

Статистика. Стили для блока статистики

```
.stats {  
  padding: 20px;  
  background: #f8f9fa;  
  border-radius: 8px;  
  display: flex;  
  gap: 30px;  
}
```

Стили для текста в статистике

```
.stats p {  
  font-size: 16px;  
  color: #555;  
}
```

Стили для числовых значений в статистике

```
.stats span {  
  font-weight: bold;  
  color: #667eea;  
  font-size: 18px;  
}
```

Смотрим результат: Обновите страницу в браузере

The screenshot shows a web application titled "Lab 24-26 - Modern JavaScript" with a subtitle "Менеджер задач". It features a form to "Добавить новую задачу" (Add new task) with a text input and a "Добавить" button. Below the form are three filter buttons: "Все", "Активные", and "Завершённые". A list of tasks is displayed, each with a checkbox and a "Удалить" button. The tasks are: "погулять" (unchecked), "поиграть" (unchecked), and "покупать" (checked). At the bottom, a summary bar shows "Всего задач: 3" and "Выполнено: 1".

13.4. Проверка работы приложения

Тестируем все функции:

1. Добавление задачи:

- Введите "Купить молоко" и нажмите "Добавить"
- Задача должна появиться в списке

2. Добавление по Enter:

- Введите "Сделать домашку" и нажмите Enter
- Задача добавится

3. Отметка выполненной:

- Кликните на чекбокс у любой задачи
- Текст должен зачеркнуться

4. Удаление:

- Нажмите "Удалить" у любой задачи
- Появится подтверждение, задача удалится

5. Сохранение:

- Добавьте несколько задач
- Обновите страницу (F5)
- Задачи должны остаться!

13.5. Что мы использовали из ES6+

В этом приложении применили все изученные концепции:

- **Деструктуризация** - в объектах задач
- **Arrow функции** - в обработчиках событий
- **Template literals** - для создания HTML (обратные кавычки)
- **Методы массивов** - `.push()`, `.find()`, `.filter()`, `.forEach()`
- **Тернарный оператор** - `task.completed ? 'completed' : ''`
- **LocalStorage** - для сохранения данных
- **JSON** - `.stringify()` и `.parse()`

13.6. Практическое задание

Задание:

1. Убедитесь, что приложение полностью работает
2. Добавьте минимум 5 разных задач
3. Отметьте 2-3 задачи как выполненные
4. Обновите страницу - задачи должны сохраниться.

Практическое задание

1. Сделайте скриншот **работающего приложения** с задачами и сохраните в папку **img** с именем:

step13_finalAppLab26_FIO.png

2. Сделайте скриншот **консоли браузера** с логами и сохраните:

step13_consoleLab26_FIO.png

3. Выполните команды:

git add .

git commit -m "Final task manager application completed"

git push

Самостоятельное задание: описание README.md

Задание: Добавьте описание в README.md.

Пример оформления:

Лабораторная работа №24-26 - Современный JavaScript (ES6+)

Выполнил: [Ваше ФИО]

Группа: [Номер группы]

Дата: [Дата выполнения]

Цель работы

Изучить и закрепить ключевые возможности современного JavaScript (ES6+):

- Деструктуризация
- Spread/Rest операторы
- Модули (Import/Export)
- Промисы и Async/Await
- Fetch API для работы с REST API
- Optional Chaining и Nullish Coalescing

Основные выводы

В ходе выполнения лабораторной работы были изучены и применены на практике ключевые возможности современного JavaScript:

1. **Деструктуризация** значительно упрощает работу с объектами и массивами
2. **Spread/Rest** операторы делают код более чистым и читаемым
3. **Модули** позволяют организовать код в больших проектах
4. **Async/Await** упрощает работу с асинхронным кодом
5. **Fetch API** - современный стандарт для HTTP-запросов
6. **Optional Chaining** предотвращает ошибки при работе с вложенными объектами

Все эти концепции являются фундаментальными для современной веб-разработки и активно используются в React, Vue, Angular и других фреймворках.

Ссылка на репозиторий

[GitHub Repository](#)

Выполните финальные команды:

git add .

git commit -m "Added README and final documentation"

git push